→ Speed Limit.

⇒ Slow Kernels.

the profiler

SM utilization measures whether threads are active, not whether they are doing useful work. A kernel can be 100% utilized and still leave 99% of the hardware's capability untouched.

SM utilization

This line is the fraction of time SMs had resident warps the profiler considered active. High utilization means the scheduler usually had something to run, not that those warps were retiring useful math every cycle.

Warps still count as active while waiting on global memory, barriers, or dependencies. That is how a kernel can show ~98% utilization here yet sit near empty on memory and compute throughput below.

Memory throughput

This is achieved HBM traffic versus what the chip can sustain. A few GB/s out of thousands means the memory subsystem is barely exercised.

Typical causes are too few concurrent global accesses, an access pattern that does not keep the bus busy, or most cycles spent off the memory path entirely. Roofline thinking links this back to bytes moved per useful operation.

Compute throughput

This compares realized FP32-style throughput to the hardware peak. Hundreds of GFLOP/s against tens of thousands means the ALUs spend almost all their time waiting, not math-bound at the ceiling.

Naive matmul reloads operands from HBM far more often than necessary. Until reuse improves (tiling, caches, tensor cores), compute counters stay far below peak no matter how occupied the SMs look.

Kernel duration

Wall-clock time is what your launch actually paid. The same arithmetic can land in single-digit milliseconds when memory traffic and parallelism match what the GPU is built to feed.

Nsight ties this row to the others: long duration with high SM activity but tiny memory and compute throughput is the signature of cycles burned on traffic and stalls, not peak-rate work.

⇒ Arithmetic Intensity.

the ratio decides memory vs. compute ceiling

AI = FLOPs / bytes loaded.

→ reflects the kernel and its access pattern

High AI: Lots of math per byte; reuse, cache-friendly blocks

Low AI: streaming many bytes to do a lil work per load

NAIVE MatMul ≈ 0.15 flops/byte

TiLED MatMul 110+ flops/byte

$2N^3$ FLOPs

same multiply-add as tiled

$12N^3$ bytes

each row of A and col of B reloaded from HBM on every inner step

= ≈0.5

FLOPs / byte

The denominator is huge. The algorithm has reuse potential — each value participates in N dot products. But the naive kernel reloads it from HBM every single time.

Every dot product reloads the same row and column from HBM — paying full memory latency each time. The bytes could be cached. They never are.

$2N^3$ FLOPs

identical compute to naive

$12N^3$ / T bytes

each tile loaded from HBM once — reused T times from shared mem

= 110+

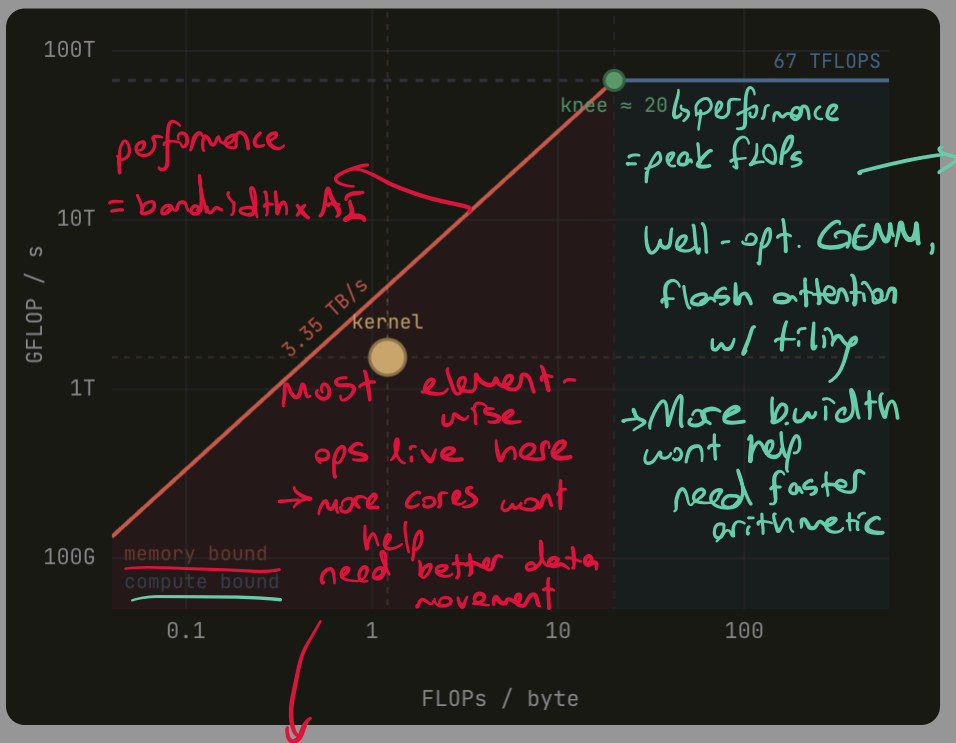
FLOPs / byte

Same FLOPs, denominator collapses. T=32 means each HBM load is reused 32 times in fast on-chip SRAM. 32× fewer bytes from slow memory → 32× higher AI.

Tiling is a denominator attack. The FLOPs stay the same. The bytes loaded shrink by T because each load is reused T times in fast shared memory. Same algorithm — completely different AI.

→ The Model.

⇒ The Diagram.



performance = bandwidth x AI

- fetching byte is bottleneck
- Math units sit idle waiting for data

+ fix: coalesce memory access, tile into shmem, increase data reuse

Most element-wise ops live here
→ more cores wont help
need better data movement

→ More bandwidth wont help
need faster arithmetic

= peak FLOPs

Well-opt. GEMM, flash attention w/ tiling

- Math units are bottleneck
- data flowing fast enough

+ fix: tensor cores, lower precision, reduce redundant FLOPs